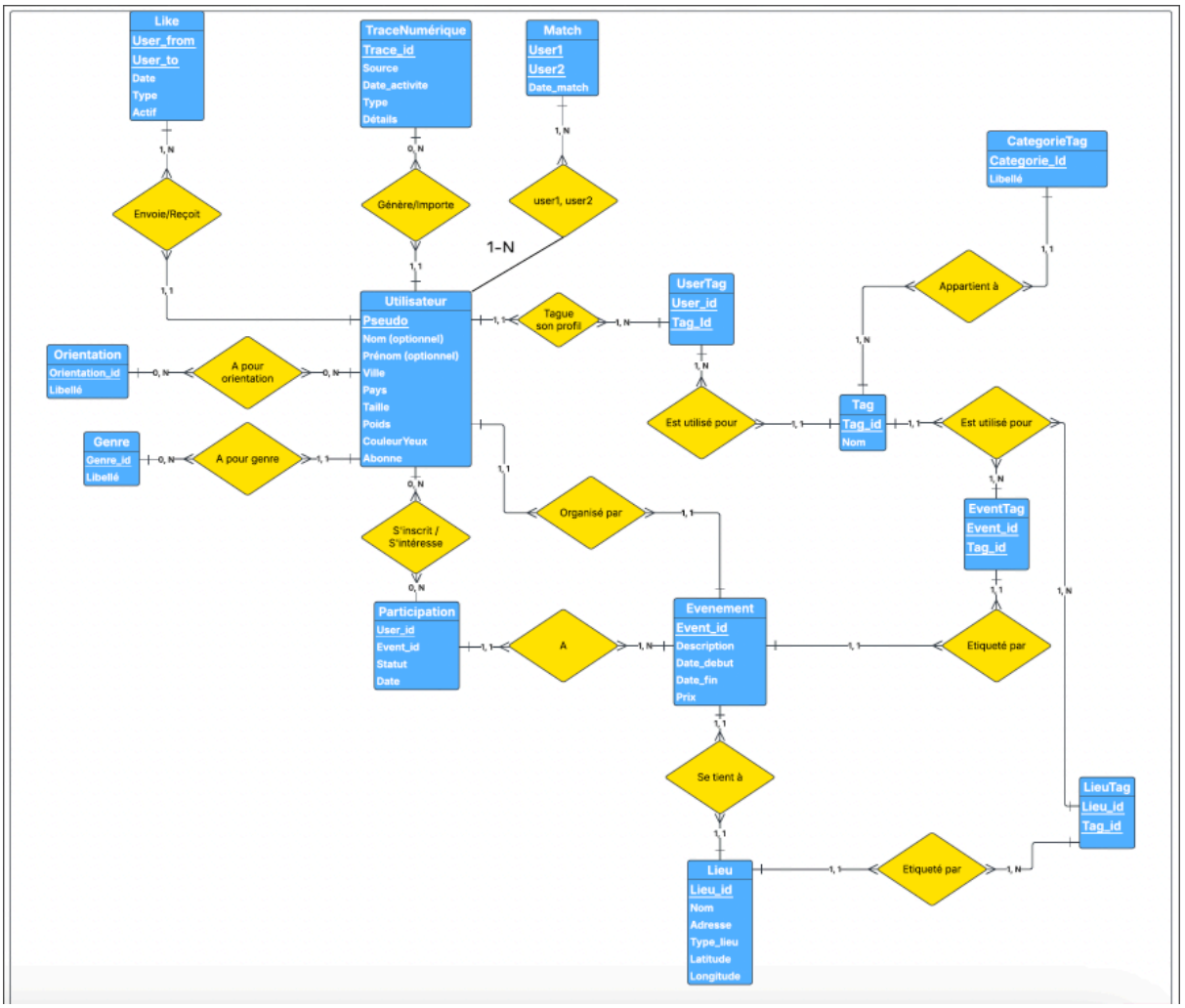
RAPPORT DE PROJET : LE BIG MATCH

EL KADIRI Youssef - IMBERT Gabriel

1) Modélisation conceptuelle



Cette figure reprend le modèle entités-associations complet élaboré pour Le Big Match : on distingue les quatorze entités principales, les associations intermédiaires et les cardinalités détaillées (0,N ; 1,N ; 1,1, etc.).

Les données d'activité (posts, achats, Strava, etc.) partent dans TraceNumérique avec une FK vers l'utilisateur, au lieu d'enfler la fiche profil. On peut ainsi purger les traces sensibles sans toucher au profil, et les insertions massives sont non bloquantes sur la table principale.

Nous avons le couple Tag <-> CategorieTag (+ parent_id auto-référent) pour gérer catégories et sous-catégories, ce qui peut permettre des requêtes hiérarchiques récursives.

On logge tous les likes/nope dans Like. Un trigger (ou job SQL) matérialise la réciprocité dans Match. On garde ainsi l'historique unilatéral, ce qui aide pour des analyses ("qui like sans retour ?"). Les requêtes sociales (trouver les matches) se font en O(1) indexé.

La table Participation est une table association (user_id, event_id) avec les attributs statut et date. Elle stocke l'historique des désinscriptions, et autorise plusieurs statuts (intéressé/confirmé/annulé).

Ce schéma comprend également quelques cardinalités :

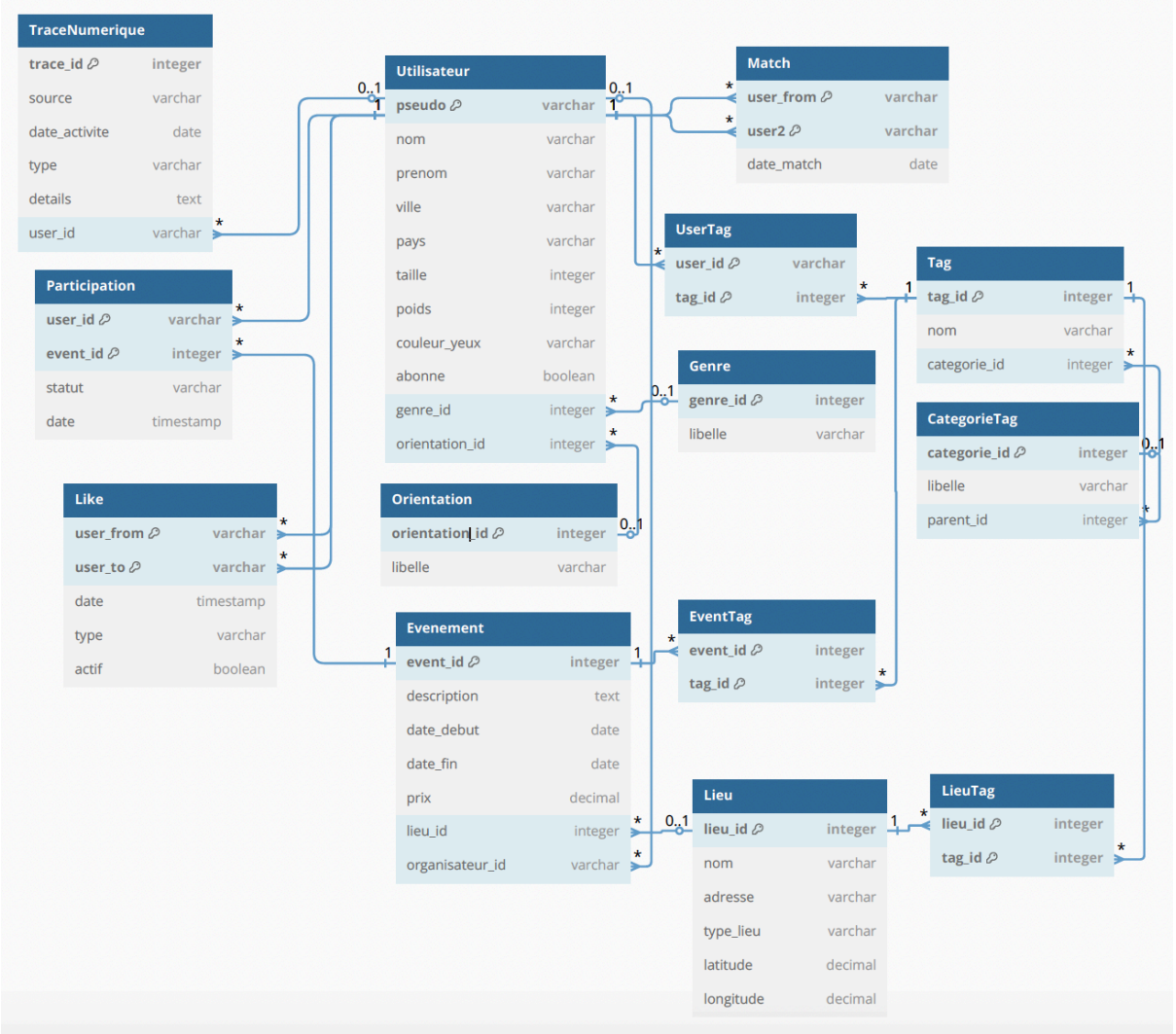
- Un Evenement est organisé par un unique Utilisateur, mais peut accueillir 0,N participants.
- Un Tag peut être lié à 0,N utilisateurs/lieux/événements ; réciproquement, chaque profil peut avoir 0,N tags.
- Un Utilisateur peut générer 0,N traces numériques.

Ces multiplicités traduisent les usages réels.

Nous avons choisi de laisser le côté applicatif gérer certaines fonctionnalités, par exemple la différence concrète de possibilité d'action entre un utilisateur abonné premium ou non, admin ou non, etc…

Nous avons essayé de faire en sorte que cette modélisation est suffisamment riche pour alimenter un moteur de recommandation multicritères, mais assez légère pour rester maniable et évolutive lors de futures features (messagerie, groupes, etc…).

2) Schéma relationnel



Ci-dessus, le diagramme complet des 14 tables et de leurs clés.

Presque dans toutes les tables, chaque attribut dépend uniquement de la clef primaire et aucune dépendance transitive ne subsiste.

- Les tables d'entité (Utilisateur, Evenement, Lieu, Genre, Orientation, CategorieTag, Tag) sont classiques ; seuls les couples (genre_id, orientation_id) dans Utilisateur font office de FKs.
- Les tables d'association (Participation, UserTag, EventTag, LieuTag, Like, Match) portent des clefs composées qui capturent la cardinalité ; ex: (user_id, event_id) dans Participation ou (user_from, user_to) dans Like.

Aucune donnée dérivée n'est stockée : l'historique des matches, le comptage des tags ou de participants est calculé à la volée. On évite ainsi les anomalies de mise à jour.

Nous avons imposé des contraintes d'intégrité majeures, notamment via des mots-clés directement dans SQL :

- **CHECK** (exemple : statut IN (‘annulé’, ‘confirm’, ‘en attente’) dans Participation ; type IN (‘like’, ‘nope’) dans Like). Cela limite la saisie à un vocabulaire contrôlé, ce qui simplifie les agrégats.
- **DROP TABLE CASCADE** : Toute FK pointe vers la PK de l’entité, sans ON DELETE CASCADE. On préfère désactiver l’entrée côté appli pour garder l’historique. Cela évite les ordres manuels.
- **ON CONFLICT DO NOTHING** : Lors de l’import des données externes, les doublons acceptés sont ignorés pour garantir l’idempotence des scripts.

L’isolement des traces numériques dans leur propre table (TraceNumerique) limite le croisement avec les données de profil tout en permettant les calculs d’affinité, l’application devrait permettre aux utilisateurs de se désengager facilement de la collecte de données grâce à TraceNumerique.

---

3) Limites de scalabilité et de conception

Le modèle tient la route pour quelques dizaines de milliers d’utilisateurs, mais il accuse vite ses limites sitôt qu’on parle de millions de profils ou de traces temps-réel.

-Traces numériques très volumineuses : TraceNumerique concentre potentiellement des millions d’entrées par jour si l’on branche Strava + Spotify + TikTok.

-Certaines règles métier (annulation de like pour abonnés uniquement, refresh du score, triggers de match réciproque) sont implémentées côté app par simplicité. Cela allège le schéma (pas de trigger pgSQL complexe), mais crée un risque d’incohérence si deux backends divergent.

4) Requêtes implémentées

Dans cette section, nous détaillons les requêtes SQL qui démontrent le bien fondé du schéma relationnel. Leur code complet est fourni dans requetes.sql. Nous présentons ici pour chacune l'objectif, les tables mobilisées et les principales techniques SQL utilisées.

Au total, 20 requêtes couvrent tous les prérequis du cahier des charges du projet.

R1 — Top organisateurs
Classer les utilisateurs par nombre d'événements créés, de participants attirés et de tags distincts utilisés.
Tables : Evenement, Participation, EventTag, Utilisateur
Techniques : agrégats, LEFT JOIN, GROUP BY, ORDER BY
Utilisations possibles dans l'application: Identifier les organisateurs populaires pour recommander leurs événements aux utilisateurs.

R2 — Centres d'intérêt communs (>= 7)
Déceler les binômes d'utilisateurs partageant au moins sept tags.
Tables : UserTag, Utilisateur
Techniques : auto-jointure, HAVING
Utilisations possibles dans l'application: Proposer des mises en relation entre utilisateurs partageant au moins 7 intérêts.

R3 — Co-participations à des événements (>= 2)
Suggérer des rencontres entre personnes qui se croisent souvent.
Tables : Participation, Utilisateur
Techniques : double jointure sur la même table + filtre d'état 'confirmé'
Utilisations possibles dans l'application: Proposer des rencontres entre utilisateurs ayant participé à au moins deux mêmes événements.

R4 — Evénements premium
Identifier les événements dont le prix excède de 20% la moyenne de l'organisateur.
Table : Evenement
Techniques : sous-requête corrélée
Utilisations possibles dans l'application:Promouvoir des événements haut de gamme auprès des utilisateurs premium.
Pourquoi avoir utilisé une sous requête:

La comparaison ici dépend à la fois du prix de la ligne courante, et d'un agrégat (c'est-à-dire ici la moyenne de tous les cousins du même organisateur), qui est calculé sur plusieurs lignes. Une simple jointure en mode JOIN + GROUP BY ne suffirait pas ici : si on voulait le faire, on devrait d'abord construire une table "moyenne par organisateur" (sinon la jointure duplique chaque événement pour toutes les autres lignes de son organisateur). Cette table agrégée serait déjà une sous-requête dans le FROM ou un CTE : on n'a donc rien "économisé", on a juste déplacé le problème.

R5 — 10 profils les plus compatibles
Proposer à llefebvre13 les dix profils partageant le plus de tags, en évitant doublons de like/match.
Tables : UserTag, Like, Match, Utilisateur
Techniques : sous-requête + jointures externes + LIMIT
Utilisations possibles dans l'application:Proposer 10 profils les plus compatibles via intérêts communs.
Pourquoi avoir utilisé une sous requête:
La logique ici implique deux choses : une agrégation par candidat (nombre de tags communs) et un filtrage après coup (likes / matches). Sans sous-requête, il faudrait un GROUP BY sur toute la requête, or on doit encore y ajouter deux LEFT JOIN (Like, Match). On obtiendrait soit des doublons, soit une requête assez illisible. Autrement dit, la jointure seule ne suffit pas à produire le résultat voulu sans redondance.

R6 — Profils les plus likés (>35)
Repérer les utilisateurs qui génèrent le plus d'intérêt.
Table : Like
Techniques : GROUP BY … HAVING
Utilisations possibles dans l'application:Mettre en avant les profils les plus sollicités par la communauté.

R7 — Événements les plus populaires (>= 45 confirmations)
Mettre en avant les événements à forte affluence.
Tables : Participation, Evenement
Techniques : agrégat + HAVING
Utilisations possibles dans l'application:Mettre en avant les événements les plus populaires aux utilisateurs.

R8 — Record mensuel de likes par utilisateur

Calculer pour chaque profil son meilleur mois et comparer à la moyenne globale.
Table : Like
Techniques : CTE + agrégat imbriqué + ROUND
Utilisations possibles dans l'application:Comparer les meilleurs mois de likes des utilisateurs pour animer un classement mensuel.

R9 — Occupation des lieux (0 → N événements)
Lister tous les lieux, y compris ceux encore inoccupés.
Tables : Lieu, Evenement
Techniques : LEFT JOIN + comptage
Utilisations possibles dans l'application:Lister tous les lieux, même inactifs, pour proposer de nouveaux spots de rencontre.
Interet de LEFT_JOIN: On veut garder les lieux qui n'ont pas d'evenements

R10 — Organisateurs toujours gratuits (EXISTS)
Trouver les organisateurs n'ayant jamais fait payer.
Tables : Evenement, Utilisateur
Techniques : double sous-requête EXISTS / NOT EXISTS
Utilisations possibles dans l'application:Mettre en avant les organisateurs d'événements gratuits.

R11 — Organisateurs toujours gratuits (agrégat)
Variante de R10 via GROUP BY + MAX(prix).

R12 — Activité numérique (compte simple)
Classer les profils par volume de traces numériques (source non null).
Table : TraceNumerique
Techniques : agrégat simple
Utilisations possibles dans l'application:Identifier les utilisateurs les plus actifs pour leur proposer des fonctionnalités avancées.

R13 — Activité numérique (filtre d'agrégat)
Variante de R12 utilisant l'opérateur FILTER pour un code plus expressif.

R14 — Prochaine date libre dans les lieux très demandés
Générer récursivement la première date disponible pour les lieux qui accueillent déjà >3 événements.
Table : Evenement
Techniques : CTE récursive WITH RECURSIVE

Utilisations possibles dans l'application:Trouver le prochain jour libre pour organiser un rendez-vous en fonction de la disponibilité des lieux.

R15 — Top 10 "nopes" mensuels 2024
Par mois, classer les 10 comptes ayant reçu le plus de refus et agréger leurs tags.
Table : CategorieTag
Techniques : auto-jointure sur parent_id
Utilisations possibles dans l'application:Top 10 des utilisateurs les plus 'nopés' chaque mois.

R16 — Hiérarchie des catégories de tags
Récupérer la paire enfant/parent pour afficher la taxonomie d'intérêts.
Table : CategorieTag
Techniques : auto-jointure sur parent_id
Utilisations possibles dans l'application:Afficher la hiérarchie des intérêts dans le profil utilisateur.

R17 — Matches basés sur activité numérique identique
Rapprocher des utilisateurs partageant exactement la même activité numérique.
Table : TraceNumerique
Techniques : jointure triple clé (source, type, détails) + DISTINCT
Utilisations possibles dans l'application:Mettre en relation des utilisateurs ayant réalisé la même action sur l'application.

R18 — Dernière activité de chaque utilisateur
Mesurer la fraîcheur de l'engagement.
Table : TraceNumerique
Techniques : MAX(date_activite)
Utilisations possibles dans l'application:identifier les utilisateurs inactifs pour relancer leur engagement.

R19 — Rencontres numériques quasi temps-réel
Repérer les paires ayant répété 2x la même activité dans un intervalle de + de 3 jours.
Table : TraceNumerique
Techniques : auto-jointure temporelle + agrégat HAVING
Utilisations possibles dans l'application:Suggérer des profils ayant interagi de façon similaire en quasi-temps réel.

R20 — Proposition de “mentors numériques”
Associer chaque nouveau (< 30 jours sur l’application) à un vétéran (> 90 jours sur l’application) partageant >= 3 activités.
Table : TraceNumerique
Techniques : deux CTE (nouveaux, experimente) + jointures multiples
Utilisations possibles dans l’application:Associer nouveaux et mentors selon activités partagées.

—----------

5) **Indice de recommandation**

L’indice score_profil(u_cible, candidat) additionne sept critères, pondérés de 0 à 100 points :

- **Tags communs :** 30%
- **Co-participation à des événements :** 20%
- **Likes directs (unilatéraux) :** 15%
- **Likes mutuels :** 15%
- **Même ville :** 10%
- **Même pays (ville différente) :** 10%

Chaque sous-score est borné entre 0 et 1 puis multiplié par son poids, ce qui produit un total sur 100. De même, il existe un critère obligatoire : on ne doit proposer à l’utilisateur que des personnes possédant la même orientation, pour que sa recherche corresponde réellement à ce dont il a envie. Le score se lit comme un pourcentage d’affinité.

Pourquoi ce modèle ?

Dans les modèles hybrides (Netflix, Spotify, Tiner), on sépare ce que l’utilisateur est/a fait de ce qu’il veut faire maintenant. Les tags communs et événements déjà partagés représentent la proximité thématique et la preuve d’intérêt passée. Ces signaux sont stables et exploitables, même si l’utilisateur vient d’arriver (il déclare ses tags) ou n’a encore liké personne. Les autres données captent la dynamique en temps réel : envie exprimées (“j’aime ce profil”), faisabilité logistique (être dans la même ville) et sécurité sociale (orientation).

L’indice score_event(u_cible, evenement) équilibre affinité, historique et contexte :

- **Affinité tags :** 30%
- **Historique participations similaires :** 30%
- **Popularité :** 20%
- **Proximité géographique :** 15%
- **Présence de profils compatibles :** 5%

Comme pour les profils, chaque composante renvoie un score normalisé [0,1] avant pondération.
Ce modèle suit la même logique que le précédent.

6) Conclusion et pistes futures

Ce projet possède un modèle entités-associations, des requêtes “features clés” qui démontrent l’utilité concrète du schéma, ainsi qu’un scoring unifié lisible et paramétrable. En combinaison, ces briques fournissent une base pour alimenter une appli de rencontres qui privilégie la transparence et la performance.
A court terme, plusieurs améliorations sont possibles, notamment, en premier lieu, les triggers de tenue de concordance. On pourrait aussi passer de comparaisons ville/pays à un vrai calcul de distance, ce qui permettra un tri par “rayon”, déverrouillant la recherche “événement à moins de 3 km”. Enfin, par simple principe d’anonymisation et de consentement RGPD, on pourrait chiffrer des colonnes sensibles (et surtout les traces GPS évoquées précédemment), ainsi qu’implémenter des procédures pour supprimer les entrées de TraceNumerique sans toucher aux métriques agrégées.